

Constraint-Safe Quantum-Guided Large-Neighborhood Search for Sparse Multi-Asset Portfolios

Technical report

August 2026

Abstract

Sparse portfolio construction combines a convex allocation problem with a combinatorial support-selection problem. The distinction matters in practice: an attractive subset of assets can become infeasible after turnover, group, position, factor, or stress constraints are imposed, while a feasible subset may still receive poor continuous weights. This study implements a hybrid method that separates these decisions. A factor-based quadratic program first solves the full-universe relaxation. Classical large-neighborhood search and a fixed-cardinality XY-QAOA circuit then propose local changes to the support. Every proposed support is reallocated by the same continuous financial model and checked by an independent validator. A direct Gurobi mixed-integer quadratic program supplies an incumbent and lower bound on instances where certification is computationally appropriate.

On the 2,000-asset reference instance, the method returned a valid 50-asset portfolio with no hard-constraint breaches. XY-QAOA improved the initial feasible objective by 1.87%; the time-limited Gurobi result remained 2.97% better than the quantum-guided incumbent and reported a 0.00224% MIP gap. The result supports a practical conclusion rather than a quantum-advantage claim: small quantum neighborhoods can be embedded safely in a scalable classical allocation system, while classical optimization remains responsible for exact weights, validation, and certification. A matrix-free factor representation extends the same search protocol to 20,000 assets without forming dense covariance or correlation matrices.

1. Portfolio construction problem

Let n denote the number of eligible assets. The decision variables are the new portfolio weights $w \in \mathbb{R}^n$ and binary support indicators $z \in \{0, 1\}^n$. The current portfolio is w^0 . Expected return, income yield, and linear trading-cost coefficients are denoted by μ , y , and c , respectively. The covariance matrix is Σ .

The implemented objective is

$$\min_{w, z, t} \quad \lambda_r w^T \Sigma w - \lambda_g \mu^T w - \lambda_y y^T w + \lambda_c c^T t, \quad (1)$$

where $t_i \geq |w_i - w_i^0|$. The four terms represent variance, expected growth, income, and implementation cost. Equation (1) is a ranking objective; its value is not a return percentage.

The main benchmark enforces

$$\begin{aligned}
\mathbf{1}^T w &= 1, & w &\geq 0, \\
m_i z_i &\leq w_i \leq u_i z_i, \\
\sum_{i=1}^n z_i &= K, \\
L_g &\leq \sum_{i \in g} w_i \leq U_g, \\
\sum_i |w_i - w_i^0| &\leq T_{\max}.
\end{aligned} \tag{2}$$

The first line invests the full budget. The second links selection to minimum and maximum active weights. The third fixes the number of holdings. The last two impose group allocation and turnover limits. The software can also enforce eligibility, mandatory assets, income and return floors, factor bands, scenario-specific stress floors, and empirical conditional value at risk (CVaR). These extensions are enabled only when the corresponding inputs are available; the benchmark does not manufacture constraints merely to increase model complexity.

The binary support makes (1)-(2) a mixed-integer quadratic program. Even with a positive-semidefinite covariance matrix, the continuous allocation is convex but the exact support problem is combinatorial. The hybrid method addresses this structure directly instead of discretizing all portfolio weights into binary lots.

2. Factor-native risk representation

Large universes use the factor decomposition

$$\Sigma = B\Omega B^T + D, \tag{3}$$

where $B \in \mathbb{R}^{n \times k}$ contains factor loadings, Ω is the factor covariance matrix, and D is diagonal idiosyncratic variance. Risk is evaluated as

$$w^T \Sigma w = (B^T w)^T \Omega (B^T w) + \sum_i D_{ii} w_i^2. \tag{4}$$

Equation (4) avoids dense $n \times n$ storage. One double-precision dense matrix requires approximately 0.75 GiB at 10,000 assets and 2.98 GiB at 20,000 assets; retaining both covariance and correlation doubles those values. With 12 factors, the principal factor arrays at 20,000 assets require only a few megabytes. Solver workspaces and result tables still consume additional memory, but the dominant quadratic storage term is removed.

The implementation exposes covariance matrix-vector products and small support submatrices through the factor representation. Continuous optimization, support scoring, allocation, validation, scenario generation, and presentation sampling can therefore operate when the dense matrices are absent.

3. Hybrid solution method

3.1 Continuous relaxation and valid initialization

The first stage removes exact cardinality and minimum-active-weight constraints while retaining the convex financial guardrails. Its solution provides a lower bound for the sparse minimization problem and a full-universe ranking signal.

A feasible exact- K support is then constructed. Mandatory holdings and groups with positive lower bounds are protected. Ranked deterministic trials are evaluated first. If they fail, a linear feasibility MILP jointly enforces budget, support linkage, group limits, turnover, and enabled linearized risk guardrails. The algorithm does not begin neighborhood search until it has a portfolio that passes the independent validator.

3.2 Fixed-support allocation oracle

Given a candidate support S , all weights outside S are fixed at zero and the remaining continuous convex allocation problem is solved exactly to the chosen numerical tolerance. The oracle then recomputes every hard constraint independently. A support is rejected if any violation exceeds the validation tolerance. Results are cached by support, which makes duplicate classical and quantum proposals inexpensive.

This oracle is the principal safety boundary. Neither a low surrogate energy nor a valid bitstring is sufficient for acceptance.

3.3 Adaptive large-neighborhood search

At iteration j , the current support is divided into frozen holdings and a change window W_j . The window contains weak removable holdings and promising unheld candidates. Candidate selection incorporates relaxation scores, marginal risk, group pressure, and correlation-community diversity. Previously explored unheld assets are excluded from subsequent windows when alternatives exist.

If r of the $F = |W_j|$ assets are currently held, a window proposal must satisfy

$$\sum_{i \in W_j} x_i = r. \quad (5)$$

Classical tabu/LNS ranks one-for-one swaps using the window surrogate and sends only the best distinct supports to the allocation oracle. Small windows can be enumerated exactly.

3.4 XY-QAOA window search

The same window is written as a QUBO,

$$E(x) = x^T Q x + h^T x, \quad x \in \{0, 1\}^F, \quad (6)$$

subject to (5). The cost includes interactions within the window, covariance with frozen holdings, return, income, transaction-cost changes, and group-bound pressure. It remains a search surrogate: candidate supports are rescored by the full allocation model.

The XY mixer is

$$H_M = \frac{1}{2} \sum_{(i,j) \in E} (X_i X_j + Y_i Y_j). \quad (7)$$

Each mixer term exchanges 10 and 01; it preserves Hamming weight. Starting from the current support therefore enforces (5) by circuit construction in the ideal model, avoiding a tuned cardinality penalty.

For the default 16-qubit, seven-excitation window, parameter optimization uses the exact fixed-weight subspace of

$$\binom{16}{7} = 11,440 \quad (8)$$

states on the CPU. Aer GPU samples the corresponding physical circuit after the angles are selected. This division is deliberate. Moving dozens of small, sequential COBYLA evaluations to the GPU would introduce compilation and kernel-launch overhead while discarding the compact subspace representation. The GPU is used for the task to which it is better suited: physical-circuit execution and sampling.

3.5 Classical reference and validation

The direct Gurobi formulation uses continuous weights, binary support, the factor risk model, turnover epigraph variables, all configured hard constraints, and the best hybrid portfolio as a MIP start. A time-limited run returns the incumbent, best bound, reported MIP gap, nodes, and timing. A fixed-support QP being optimal does not make LNS or QAOA globally optimal.

The validator is solver-independent and does not clip negative residuals or renormalize weights after a solve. It reports the left-hand side, right-hand side, signed slack, numerical violation, and pass/fail status of every rule.

4. Two-thousand-asset reference result

The reference instance contains 2,000 assets, 10 asset groups, 12 risk factors, and exactly 50 final holdings. Active weights lie between 0.5% and 4.0%. L1 turnover is capped at 40%, corresponding to 20% under the common one-way turnover convention. Three 16-asset, seven-excitation windows are evaluated.

Quantity	Result
Continuous-relaxation objective	-0.04610304137
Valid initial objective	-0.04390343596
Best XY-QAOA-guided objective	-0.04472303024
Gurobi incumbent	-0.04609222044
Gurobi best bound	-0.04609325222
Gurobi reported MIP gap	0.00224%
Selected assets	50
Independent hard-constraint breaches	0
Maximum numerical violation	1.17×10^{-15}
End-to-end runtime	21.588 s
Gurobi component runtime	11.262 s

XY-QAOA improved the valid initial objective by 1.87%. The Gurobi incumbent was 2.97% better than the XY-QAOA-guided incumbent and lay close to the continuous lower bound. The three ideal XY windows preserved the required seven selected assets in all recorded shots. These observations show that the quantum component generated valid and, in two windows, improving supports. They do not show a speed or quality advantage over the classical reference.

Several constraints were genuinely active: exact cardinality, the turnover cap, a group upper bound, a group lower bound, four maximum-weight positions, and four minimum-active-weight positions. This pattern is consistent with a support-constrained rebalance rather than an unconstrained mean-variance portfolio.

The synthetic out-of-sample path is also internally coherent. The Gurobi portfolio exhibited lower realized volatility, CVaR, and drawdown, while the initial portfolio ended with greater wealth on that particular path. A single path is illustrative and cannot establish statistical dominance.

5. Scaling experiment

The scaling study fixes the portfolio cardinality, factor count, quantum window size, search budget, and validation tolerance while increasing the universe size. The default grid is

$$n \in \{250, 500, 1000, 2000, 5000, 10000, 20000\}. \quad (9)$$

Each size is run with three seeds in a fresh process. The study records median and interquartile range for:

- full search time and time to first valid portfolio;
- factor-QP, initialization, classical-window, quantum-window, allocation, and certification time;
- peak resident memory and dense covariance storage avoided;

- objective gap to the continuous relaxation;
- gap to Gurobi where a comparable incumbent is available;
- zero-breach completion rate;
- quantum angle, sampler, and allocation time;
- fixed-cardinality shot rate and actual execution device.

Gurobi certification is attempted only through 2,000 assets by default. Above that threshold, the experiment measures the factor-native relaxation and hybrid search. This distinction prevents a 20,000-asset heuristic run from being presented as a certified mixed-integer optimum.

The largest default full-universe experiment is 20,000 assets. It is a deliberate engineering target rather than a theoretical maximum. A 50,000-asset factor-relaxation stress test is plausible on a server with adequate RAM, but it should be reported only after measurement and should not replace the end-to-end 20,000-asset study. Exact MIQP scale is instance- and time-limit dependent; no universal maximum can be inferred from the number of assets alone.

6. IBM QPU experiment

The asset universe and QPU width are decoupled. The QPU receives only the adaptive change window, so an 8-, 12-, or 16-qubit circuit can be tested while the classical model still contains thousands of assets. This is the relevant hardware experiment for the present architecture.

The QPU protocol transfers simulator-optimized angles, uses Qiskit Runtime `SamplerV2`, and sends the most frequent distinct bitstrings to the allocation oracle. The preferred sequence is 8, 12, then 16 qubits at depth $p = 1$, with 4,096-8,192 shots. Twenty qubits and depth $p = 2$ are extensions only when transpiled two-qubit depth and calibration quality remain acceptable.

Hardware reporting includes the backend and calibration date, job identifier, shots, transpiled depth, two-qubit operations, raw cardinality rate, QPU usage time, queue-inclusive wall time, complete end-to-end time, valid unique supports, and best allocated objective. The comparison with classical LNS must use the same window and an equal total time budget. The detailed execution protocol is provided in `docs/ibm_qpu_experiment.md`.

7. Advantages and limitations

The method has five practical advantages.

First, it preserves the continuous nature of portfolio weights. Binary quantum variables describe membership, not artificial equal lots. Second, exact cardinality is built into the XY dynamics rather than imposed through a problem-dependent penalty. Third, the allocation oracle separates exploratory support generation from financial feasibility; classical and quantum proposals are judged by the same model. Fourth, factor-native risk permits large universes without dense covariance storage. Fifth, the continuous lower bound, Gurobi bound, independent validation, raw quantum diagnostics, and artifact checksums make the result auditable.

The limitations are equally important. The model is single-period and long-only. Expected returns and covariances are estimated and may be unstable. Linear trading cost does not represent nonlinear market impact. Tax lots, shorting, leverage, and multi-period recourse are outside the present scope. The quantum circuit solves a local surrogate, not the full constrained portfolio. Hardware noise can break ideal cardinality preservation, and QPU queueing can dominate wall time. Finally, synthetic backtests validate internal behavior but do not establish live investment performance.

8. Reproducibility and development disclosure

Every reported run is defined by a Git commit, YAML configuration, random seed, package/environment manifest, raw tables, independent constraint checks, and artifact checksums. Generated plots are not treated as primary evidence when their source tables are absent.

Coding-assistant tools were used during development for code review, test scaffolding, and documentation drafting. Numerical results were produced by the repository programs and checked against solver output and independent

validation tests. The project team remains responsible for the model choices, experiments, interpretation, and submitted material.

References

1. H. Markowitz, “Portfolio Selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77-91, 1952. <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>
2. E. Farhi, J. Goldstone, and S. Gutmann, “A Quantum Approximate Optimization Algorithm,” arXiv:1411.4028, 2014. <https://arxiv.org/abs/1411.4028>
3. S. Hadfield et al., “From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz,” *Algorithms*, vol. 12, no. 2, art. 34, 2019. <https://doi.org/10.3390/a12020034>
4. B. Stellato et al., “OSQP: an operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, pp. 637-672, 2020. <https://doi.org/10.1007/s12532-020-00179-2>
5. R. T. Rockafellar and S. Uryasev, “Optimization of Conditional Value-at-Risk,” *The Journal of Risk*, vol. 2, no. 3, pp. 21-41, 2000. <https://doi.org/10.21314/JOR.2000.038>